Evolutionary Computing

HW 03

Alireza Garmsiri

Introduction

In this project, we present a novel approach to training a neural network using an Evolutionary Strategy (ES). ES is a type of optimization inspired by natural selection, where a population of candidate solutions evolves over generations to optimize a target function.

We applied this method to train a neural network for a heart disease classification problem. Unlike traditional gradient-based optimization methods (e.g., stochastic gradient descent), ES operates in a gradient-free manner, making it robust against non-convexities and discontinuities in the loss landscape.

The implementation leverages self-adaptive mutation for parameter optimization and incorporates fitness evaluation based on classification metrics like accuracy, precision, F1 score, and cross-entropy loss.

Problem Statement

Heart disease remains one of the leading causes of mortality worldwide, and early detection is crucial. Machine learning, particularly neural networks, has shown promise in predictive modeling for healthcare. The goal of this project is to train a neural network to predict the presence or absence of heart disease using a preprocessed dataset. The primary challenges include:

Efficiently optimizing network parameters.

Balancing trade-offs between accuracy, precision, and F1 score.

Avoiding overfitting to the training data.

Dataset Description

The dataset, processed_heart.csv, contains patient data, including various medical metrics such as cholesterol levels, blood pressure, and other diagnostic features. The target variable (num) represents the diagnosis outcome, with the following attributes:

Features (X):

A set of numeric attributes representing patient health indicators.

Example features include age, cholesterol level, fasting blood sugar, and resting electrocardiographic results.

Target (y):

A categorical variable indicating the presence or absence of heart disease.

The dataset is split into training (80%) and validation (20%) sets. Features are normalized using StandardScaler to ensure consistent scaling, which is critical for neural network convergence.

Neural Network Architecture

The neural network is a fully connected feed-forward network designed for multi-class classification. Its structure is as follows:

Input Layer: Accepts input data with a size equal to the number of features.

Hidden Layers:

Two fully connected layers, each followed by a ReLU activation function.

These layers capture non-linear relationships in the data.

Output Layer: A softmax activation function produces probabilities for each class.

The network's parameters (weights and biases) are optimized using the evolutionary strategy.

Key Advantages of the Architecture:

Simplicity: Suitable for small to medium-sized datasets.

Flexibility: Easily extendable to more layers or neurons if required.

Evolutionary Strategy

Overview

The ES is a population-based optimization algorithm. Unlike gradient-based methods, it does not require the computation of gradients, making it suitable for non-differentiable or noisy objective functions. The specific ES variant used is the (μ, λ) strategy with self-adaptive mutation.

Key Components

Population Initialization:

A population of candidate solutions (neural network parameter vectors) is generated randomly within a specified range.

Fitness Evaluation:

The fitness of each candidate is evaluated based on a combination of metrics:

Accuracy

Precision

F1 Score

Cross-Entropy Loss

$$\text{Fitness} = \text{Accuracy} + 0.2 \cdot \text{Precision} + 0.1 \cdot \text{F1 Score} - 0.1 \cdot \text{Loss}$$

Self-Adaptive Mutation:

Mutation step sizes are dynamically adjusted using a global and individual adjustment factor:

This ensures the population maintains diversity while allowing for convergence over generations.

Selection:

The best μ individuals are selected from the offspring (λ) based on fitness scores.

Reproduction and Replacement:

Offspring replace the parent population, and the process repeats for a fixed number of generations.

Algorithm Steps

Generate an initial population of random individuals.

Evaluate the fitness of each individual.

Mutate individuals using self-adaptive step sizes.

Select the top μ individuals to form the next generation.

Repeat until the stopping criterion (e.g., fixed generations) is met.

Implementation Details

Fitness Function

The fitness function combines multiple metrics to ensure the model balances accuracy with precision and F1 score. Cross-entropy loss is included as a penalty term to encourage better class separation.

Initialization

The population is initialized within a range of $[-1,1]$ to ensure weight diversity. Step sizes are initialized uniformly.

Mutation

Mutations are applied to parameters using Gaussian noise scaled by self-adaptive step sizes. The step sizes evolve alongside the individuals.

Selection

An elitist strategy ensures the best individuals are retained, preventing performance degradation.

Performance Metrics

The following metrics are used to evaluate the network's performance:

Accuracy: Measures the proportion of correctly classified instances.

Precision: Evaluates the fraction of relevant instances among the retrieved instances.

F1 Score: Combines precision and recall, providing a single score for imbalanced datasets.

Cross-Entropy Loss: Measures the divergence between predicted probabilities and true labels.

Results

Best Performance Over Generations

The evolutionary strategy successfully improved the fitness of the best individuals over 50 generations. Key observations include:

Steady improvement in accuracy and F1 score.

A gradual reduction in loss, indicating better class separation.

Confusion Matrix

A confusion matrix was generated on the validation set to assess the model's classification performance. The matrix highlighted areas of misclassification, offering insights into potential improvements.

Conclusion

This project demonstrated the potential of evolutionary strategies for neural network training. The combination of self-adaptive mutation, fitness evaluation, and population-based optimization resulted in a robust method for optimizing network parameters. With further improvements, ES could become a competitive alternative to traditional optimization methods for machine learning.